

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении практической работы №3
по дисциплине «Сети ЭВМ и телекоммуникации
компьютерных сетей»
Вариант 8

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Мулюха В. А. _____

«_____» _____ 2026г.

Санкт-Петербург, 2026

1 Постановка задачи

Используя Docker создать контейнеры, необходимые для реализации обмена сообщениями с использованием RabbitMQ, а также показать, как именно осуществляется передача сообщений в заданных условиях.

Организовать взаимодействие трёх отправителей и трёх получателей, при этом каждому получателю соответствует собственная очередь. Отправители должны иметь возможность отправлять сообщения в каждую из очередей, используя механизм Topic Exchange с различными routing key.

2 RabbitMQ

RabbitMQ — это брокер сообщений, реализующий протокол AMQP и предназначенный для организации обмена сообщениями между различными сервисами и приложениями.

Принцип работы RabbitMQ:

1. Producer (отправитель) отправляет сообщение в Exchange.
2. Exchange принимает сообщение и определяет, в какую очередь его необходимо направить.
3. Routing осуществляется с помощью Bindings.
4. Сообщение помещается в Queue. Consumer (получатель) получает сообщение из очереди.

2.1 Topic Exchange

Topic Exchange выполняет маршрутизацию сообщений на основе routing key.

Routing key представляет собой строку, состоящую из слов, разделённых точками, например, tech.ai, news.world, sport.football.

Для сопоставления routing key используются специальные шаблоны:

- * — соответствует ровно одному слову;
- # — соответствует нулю или более слов.

3 Docker

Docker — это платформа контейнеризации, позволяющая запускать приложения в изолированных контейнерах вместе со всеми необходимыми зависимостями.

3.1 Docker Compose

Docker Compose используется для запуска многоконтейнерных приложений.

С помощью файла `docker-compose.yml` описываются:

- сервисы;
- сети;
- переменные окружения;
- параметры запуска контейнеров.

Листинг 1. `docker-compose.yml`

```
1 version: '3.9'
2
3 services:
4
5   rabbitmq:
6     image: rabbitmq:3-management
7     container_name: rabbitmq
8
9     ports:
10      - "5672:5672"
11      - "15672:15672"
12
13   alice:
14     build: ./receiver
15
16     environment:
17       AMQP_URL: amqp://guest:guest@rabbitmq:5672/
18       QUEUE_NAME: queue_alice
19       RECEIVER_NAME: Alice
20       BINDING_KEY: sport.*
21
22   bob:
23     build: ./receiver
24
25     environment:
26       AMQP_URL: amqp://guest:guest@rabbitmq:5672/
27       QUEUE_NAME: queue_bob
28       RECEIVER_NAME: Bob
29       BINDING_KEY: news.*
30
31   eva:
32     build: ./receiver
33
```

```

34     environment:
35         AMQP_URL: amqp://guest:guest@rabbitmq:5672/
36         QUEUE_NAME: queue_eva
37         RECEIVER_NAME: Eva
38         BINDING_KEY: tech.*
39
40     fred:
41         build: ./sender
42
43         environment:
44             AMQP_URL: amqp://guest:guest@rabbitmq:5672/
45             SENDER_NAME: Fred
46             ROUTING_KEYS: '["sport.football", "tech.ai"]'
47
48     simon:
49         build: ./sender
50
51         environment:
52             AMQP_URL: amqp://guest:guest@rabbitmq:5672/
53             SENDER_NAME: Simon
54             ROUTING_KEYS: '["news.world", "tech.ml"]'
55
56     trevor:
57         build: ./sender
58
59         environment:
60             AMQP_URL: amqp://guest:guest@rabbitmq:5672/
61             SENDER_NAME: Trevor
62             ROUTING_KEYS: '["sport.basketball", "news.politics"]'

```

После этого все контейнеры могут быть запущены одной командой: `docker compose up -build`

3.2 Dockerfile

Dockerfile представляет собой текстовый файл с инструкциями для сборки Docker-образа.

Листинг 2. Dockerfile

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 CMD ["python", "-u", "sender.py"]

```

Для получателя необходимо заменить строку: `CMD ["python u "sender.py"]` на `CMD ["python u "receiver.py"]`.

4 Отправитель и получатель

Механизм работы системы:

1. отправитель подключается к RabbitMQ;
2. создаётся Topic Exchange;
3. сообщение публикуется с определённым routing key;
4. Exchange анализирует routing key;
5. сообщение направляется в подходящую очередь;
6. получатель получает сообщение из своей очереди.

Одно сообщение может быть направлено одновременно в несколько очередей, если routing key соответствует нескольким шаблонам.

Листинг 3. receiver.py

```
1 import os
2 import time
3 import pika
4
5 AMQP_URL = os.getenv("AMQP_URL")
6 QUEUE_NAME = os.getenv("QUEUE_NAME")
7 RECEIVER_NAME = os.getenv("RECEIVER_NAME")
8 BINDING_KEY = os.getenv("BINDING_KEY")
9
10 EXCHANGE_NAME = "topic_exchange"
11
12
13 def connect():
14     while True:
15         try:
16             connection = pika.BlockingConnection(
17                 pika.URLParameters(AMQP_URL)
18             )
19             return connection
20         except:
21             print(f"[{RECEIVER_NAME}] Waiting RabbitMQ...")
22             time.sleep(2)
23
24
25 connection = connect()
26 channel = connection.channel()
27
28 channel.exchange_declare(
29     exchange=EXCHANGE_NAME,
30     exchange_type="topic"
31 )
32
33 channel.queue_declare(queue=QUEUE_NAME)
34
35 channel.queue_bind(
36     exchange=EXCHANGE_NAME,
```

```

37     queue=QUEUE_NAME,
38     routing_key=BINDING_KEY
39 )
40
41 print(f"[{RECEIVER_NAME}] Waiting messages...")
42
43
44 def callback(ch, method, properties, body):
45     print(
46         f"[{RECEIVER_NAME}] Received: "
47         f"{body.decode()} | key={method.routing_key}"
48     )
49
50
51 channel.basic_consume(
52     queue=QUEUE_NAME,
53     on_message_callback=callback,
54     auto_ack=True
55 )
56
57 channel.start_consuming()

```

Листинг 4. sender.py

```

1 import os
2 import json
3 import time
4 import pika
5
6 AMQP_URL = os.getenv("AMQP_URL")
7 SENDER_NAME = os.getenv("SENDER_NAME")
8
9 ROUTING_KEYS = json.loads(
10     os.getenv("ROUTING_KEYS")
11 )
12
13 EXCHANGE_NAME = "topic_exchange"
14
15
16 def connect():
17     while True:
18         try:
19             connection = pika.BlockingConnection(
20                 pika.URLParameters(AMQP_URL)
21             )
22             return connection
23         except:
24             print(f"[{SENDER_NAME}] Waiting RabbitMQ...")
25             time.sleep(2)
26
27
28 connection = connect()
29 channel = connection.channel()
30
31 channel.exchange_declare(
32     exchange=EXCHANGE_NAME,
33     exchange_type="topic"
34 )
35
36 for index, key in enumerate(ROUTING_KEYS, start=1):
37

```

```
38     message = f"Message #{index} from {SENDER_NAME}"
39
40     channel.basic_publish(
41         exchange=EXCHANGE_NAME,
42         routing_key=key,
43         body=message
44     )
45
46     print(
47         f"[{SENDER_NAME}] Sent: "
48         f"{message} | key={key}"
49     )
50
51     time.sleep(1)
52
53 connection.close()
```


5 Результаты работы

Для сборки и запуска контейнеров использовалась команда: `docker compose up -build`.

Для просмотра логов использовалась команда: `docker compose logs -f`.
Были созданы:

- контейнер RabbitMQ;
- три контейнера отправителей;
- три контейнера получателей.

В логах работы контейнеров наблюдалась успешная передача сообщений между отправителями и получателями.

Листинг 5. Пример логов

```
1 simon-1 | [Simon] Sent: Message #1 from Simon | key=news.world
2 trevor-1 | [Trevor] Sent: Message #1 from Trevor | key=sport.basketball
3 alice-1 | [Alice] Received: Message #1 from Trevor | key=sport.basketball
4 simon-1 | [Simon] Sent: Message #2 from Simon | key=tech.ml
5 trevor-1 | [Trevor] Sent: Message #2 from Trevor | key=news.politics
6 bob-1 | [Bob] Received: Message #2 from Trevor | key=news.politics
7 fred-1 | [Fred] Sent: Message #1 from Fred | key=sport.football
8 alice-1 | [Alice] Received: Message #1 from Fred | key=sport.football
```

Также работа системы была проверена с помощью веб-интерфейса RabbitMQ: <http://localhost:15672>.

Queues

► All queues (3)

Overview					Messages			Message rates			+/-
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	queue_alice	classic		running	0	0	0	0.00/s	0.00/s	0.00/s	
/	queue_bob	classic		running	0	0	0	0.00/s	0.00/s	0.00/s	
/	queue_eva	classic		running	0	0	0	0.00/s	0.00/s	0.00/s	

Рис. 1. Очереди

Заключение

В ходе лабораторной работы были созданы Docker-контейнеры для организации обмена сообщениями с использованием RabbitMQ и механизма Topic Exchange.

Были реализованы три отправителя и три получателя, взаимодействующие через отдельные очереди. С помощью routing key была реализована маршрутизация сообщений между контейнерами.

Работоспособность системы подтверждена логами запуска контейнеров и средствами мониторинга RabbitMQ.